MATH 8090: Autoregressive Integrated Moving Average (ARIMA) Models and Seasonal ARIMA Models

Whitney Huang, Clemson University

10/5-10/9/2026

Contents

ARIMA: transformations, differencing, and stochastic trends . . . 2
Monthly Price of Oil: January 1986–January 2006 . . . 2
Compare levels, changes, and proportional changes . . . 3
An example requiring two differences . . . 4
Explosive growth versus an integrated process . . . 6
A smooth curve fitted to an integrated series . . . 8
Compare deterministic and stochastic trends . . . 10
Integration means accumulating changes . . . 11
Too little and too much differencing . . . 12
Forecast changes, then reconstruct levels . . . 14
Seasonal dependence and SARIMA . . . 15
Fixed seasonal patterns versus stochastic seasonal dependence . . . 15
Seasonal AR: dependence on last year's observation . . . 16
Seasonal MA: dependence on a past seasonal shock . . . 17
Combine ordinary and seasonal integration . . . 18
Monthly International Airline Passenger Data . . . 19
Load the data and inspect its time index . . . 19
Stabilize variability with a log transformation . . . 20
Define the training period and inspect dependence . . . 21
Compare ordinary and seasonal differencing . . . 22
Assess the three differencing choices . . . 22
Develop candidates using one ordinary difference . . . 24
A helper for residual diagnostics . . . 25
Candidate 1: nonseasonal and seasonal AR terms . . . 26
Candidate 2: remove the nonseasonal AR term . . . 27

Fit integrated models for forecasting the held-out year 28
Identify the forecast horizon 29
Calculate log-level forecasts and prediction intervals 29
Plot forecasts on the log and original scales 29
Evaluate forecasts on the held-out year 31
What to remember 32

ARIMA: transformations, differencing, and stochastic trends

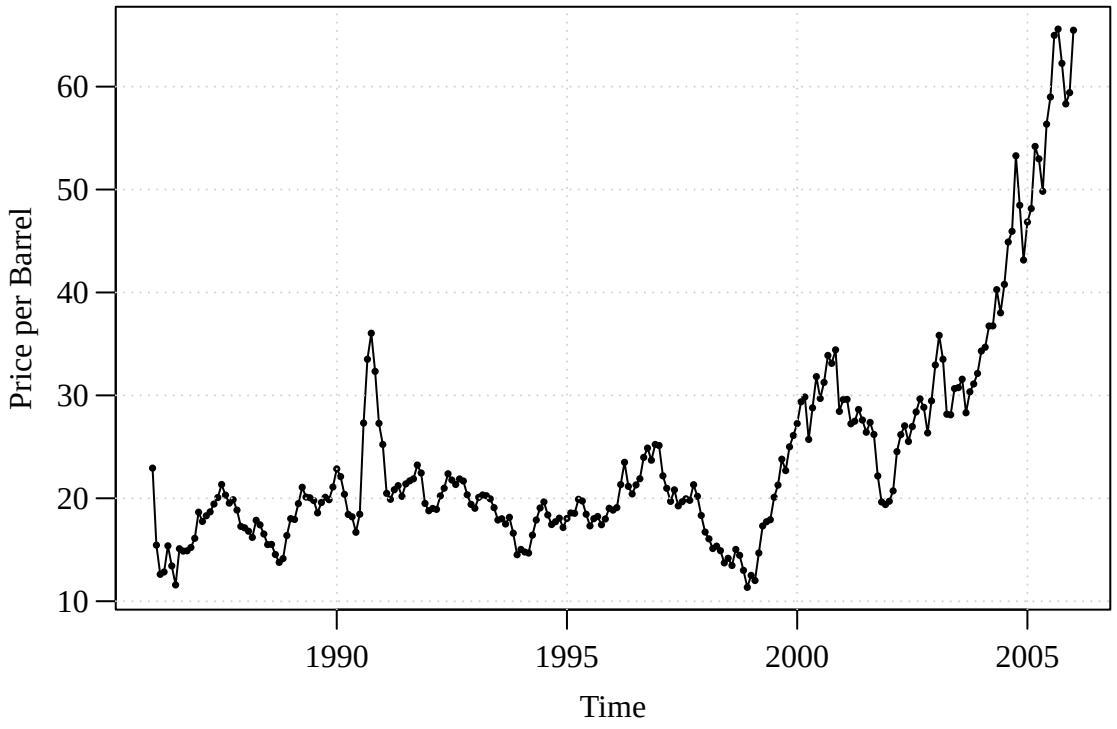
This session connects the lecture definitions to simulated and observed time series. We first examine how transformations and differencing change a series, then compare deterministic and stochastic trends. We finish with seasonal models, residual diagnostics, and forecasts for a held-out year.

Packages needed: `TSA`, `astsa`, and `mgcv`, together with `knitr` and `rmarkdown` for rendering. Numerical examples use fixed seeds for reproducibility.

Monthly Price of Oil: January 1986–January 2006

```
data(oil.price)

par(las = 1, mgp = c(2, 0.7, 0), mar = c(3.5, 3.5, 0.8, 0.6), family = "serif")
plot(oil.price, ylab = 'Price per Barrel', type = 'l')
points(oil.price, pch = 16, cex = 0.5)
grid()
```

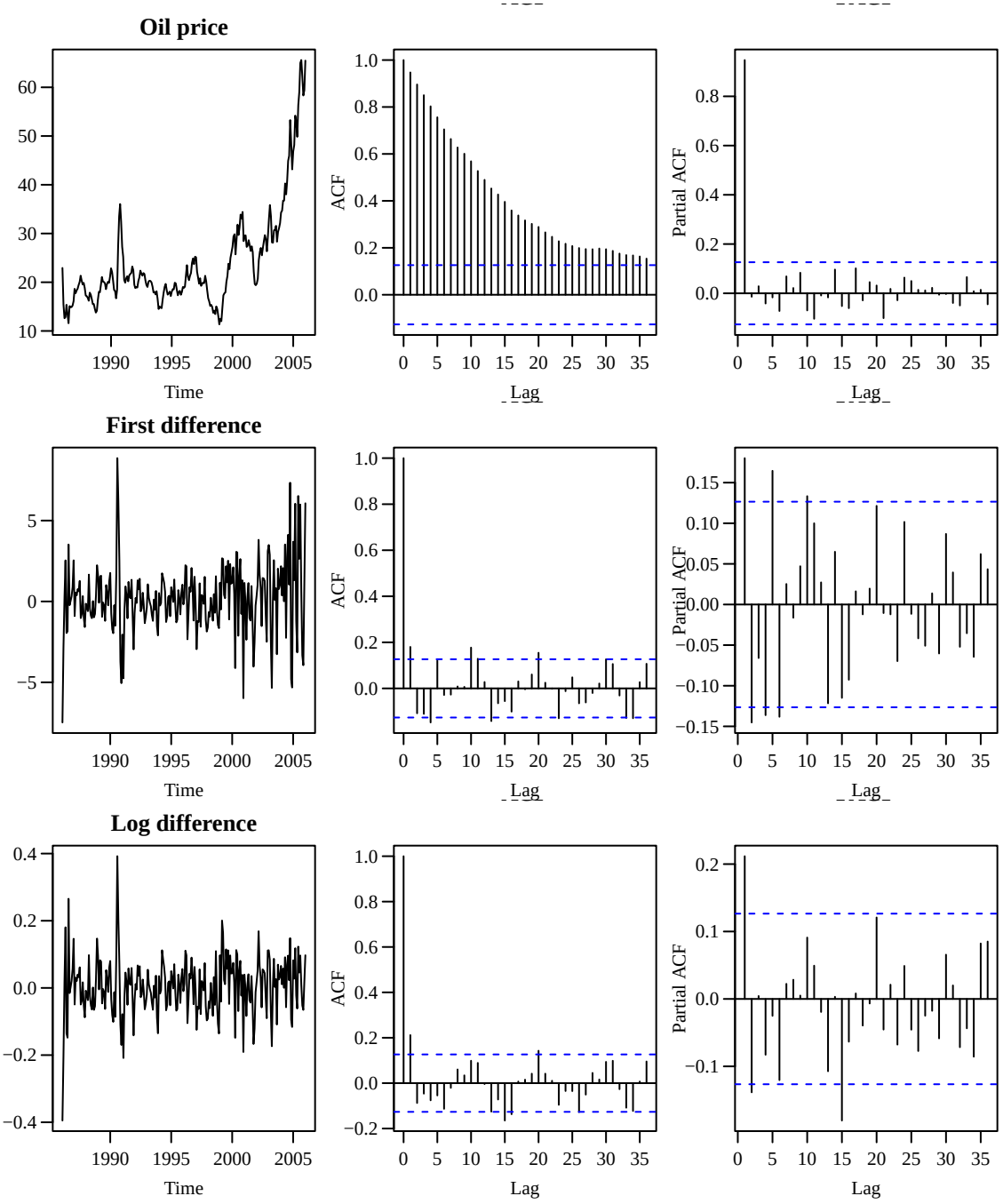


The oil-price series does not appear to fluctuate around a stable level, and a single deterministic trend is not an obvious description. Compare the original series, its first differences, and its log differences. These plots motivate candidate transformations; they do not by themselves establish stationarity.

Compare levels, changes, and proportional changes

```r
# Log differences approximate proportional changes for positive data.
oil.diff <- diff(oil.price)
oil.log.diff <- diff(log(oil.price))
series <- list("Oil price" = oil.price,
               "First difference" = oil.diff,
               "Log difference" = oil.log.diff)

par(mfrow = c(3, 3), las = 1, mar = c(3, 3, 2, 0.5), mgp = c(2, 0.7, 0), family = "serif")
for (label in names(series)) {
  x <- series[[label]]
  plot(x, type = "l", main = label, xlab = "Time", ylab = "")
  # Use observation lags, rather than time-series frequency units.
  stats::acf(as.numeric(x), main = "ACF", lag.max = 36)
  stats::pacf(as.numeric(x), main = "PACF", lag.max = 36)
}
```



Inspect the stability of the level and variability, together with the decay of the ACF. Differencing addresses a changing level; taking logarithms changes the scale and can help when variability increases with the level.

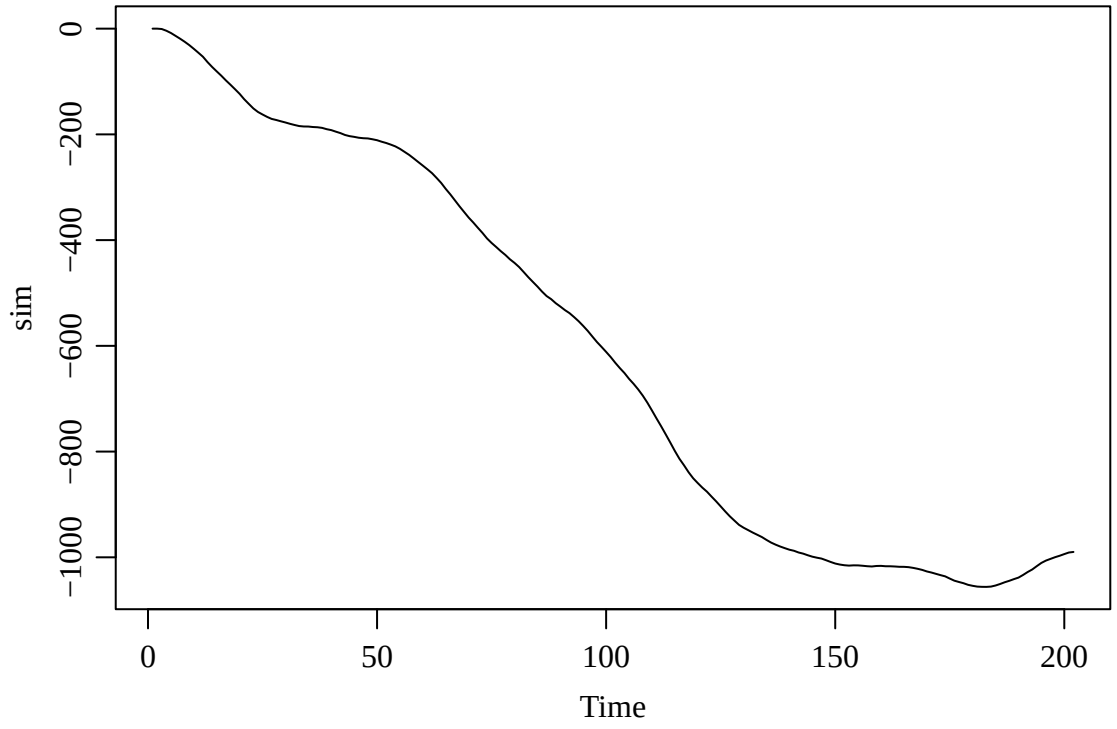
An example requiring two differences

The next simulation is ARIMA(1, 2, 0). Its second difference follows a stationary AR(1), so one difference alone is not enough.

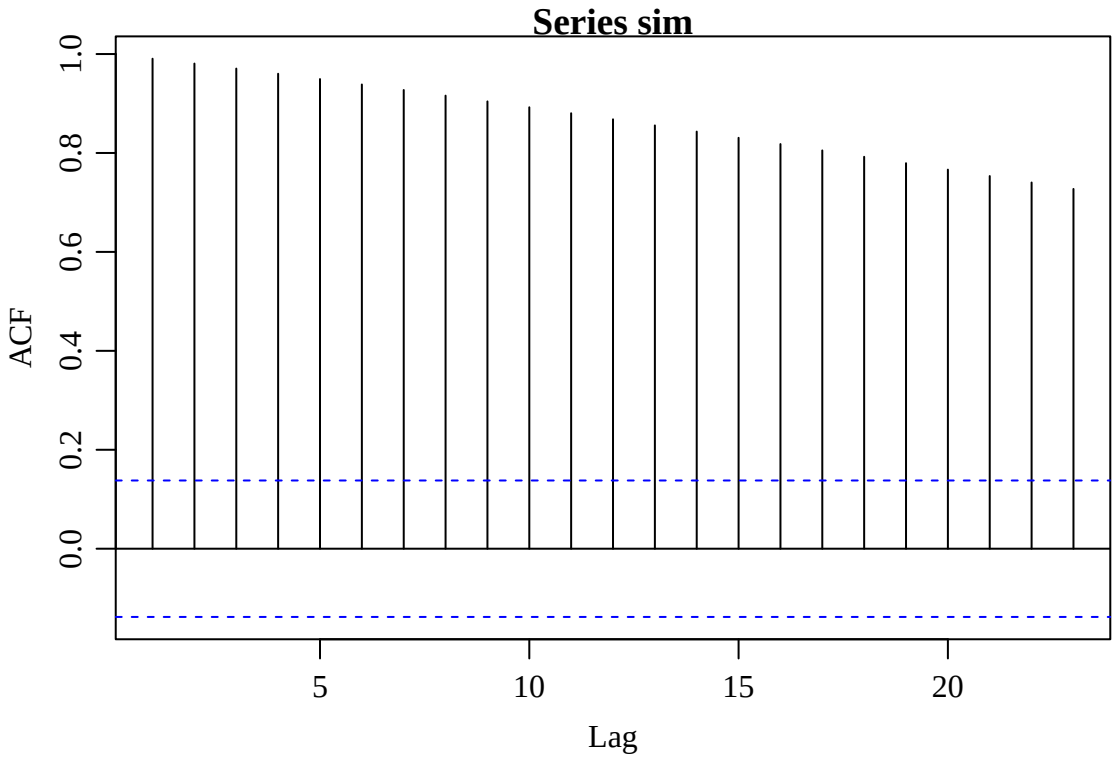
```
set.seed(2026)
par(mfrow = c(1, 1), mar = c(3.5, 3.5, 0.8, 0.6), mgp = c(2, 0.7, 0), family = "serif")
```

```r
sim <- arima.sim(list(order = c(1, 2, 0), ar = 0.1), n = 200)

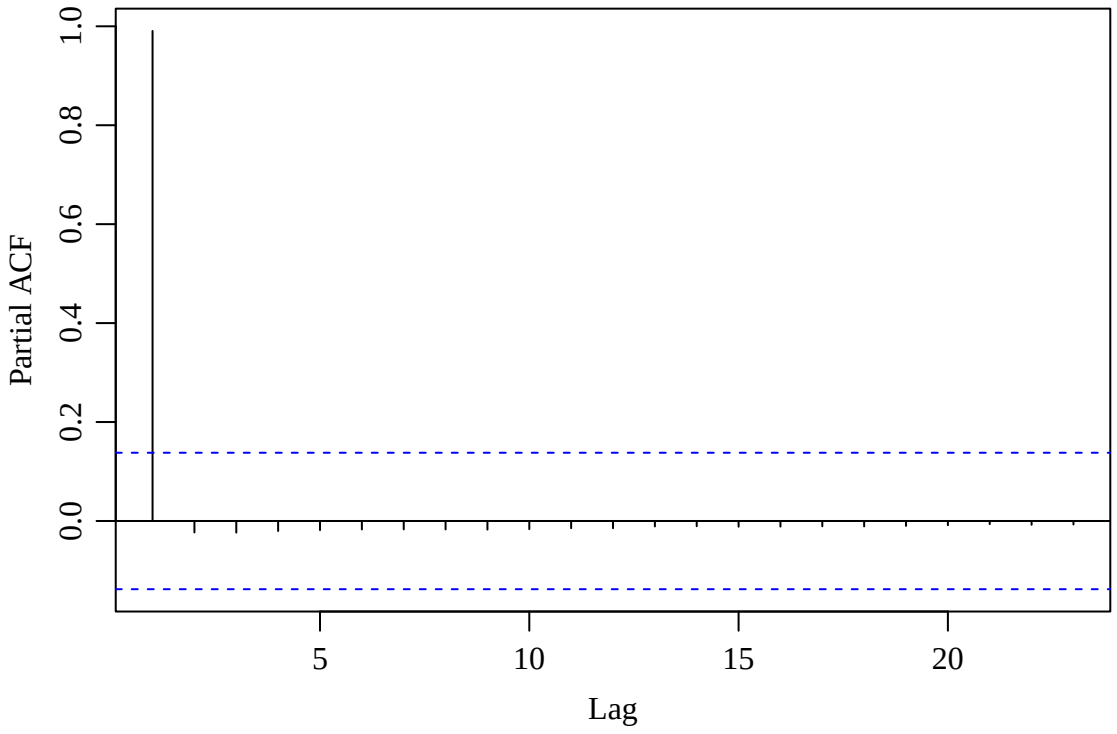
ts.plot(sim)
```




```r
acf(sim)
```



```r
pacf(sim)
```

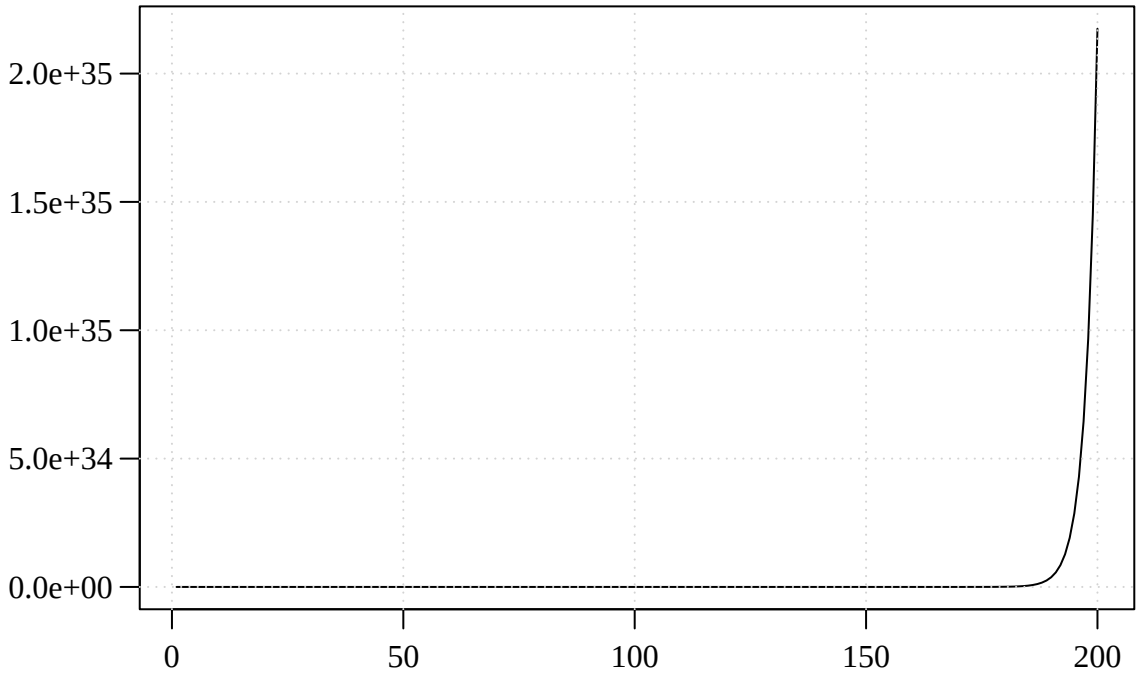


Explosive growth versus an integrated process

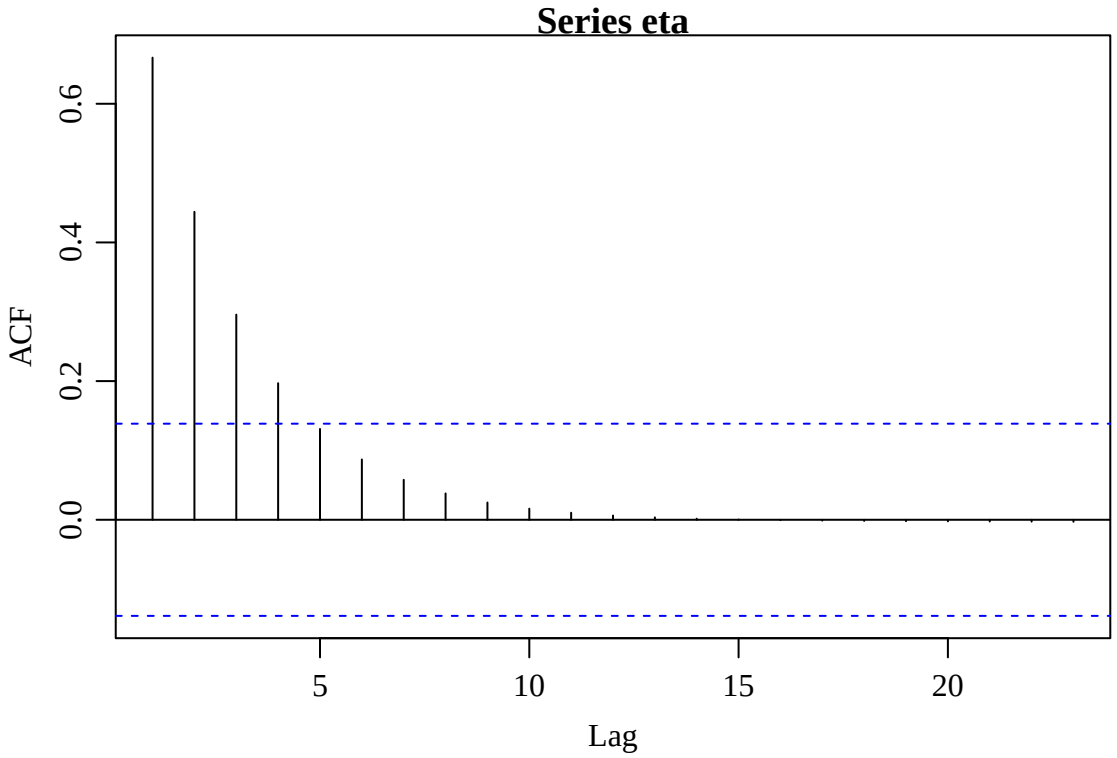
An explosive AR model and an integrated model are both nonstationary, but for different reasons. Here the explosive coefficient is 1.5. In the ARIMA(1, 1, 0) example, the coefficient 0.5 describes stationary **changes**.

$\eta_t = 1.5\eta_{t-1} + Z_t$.

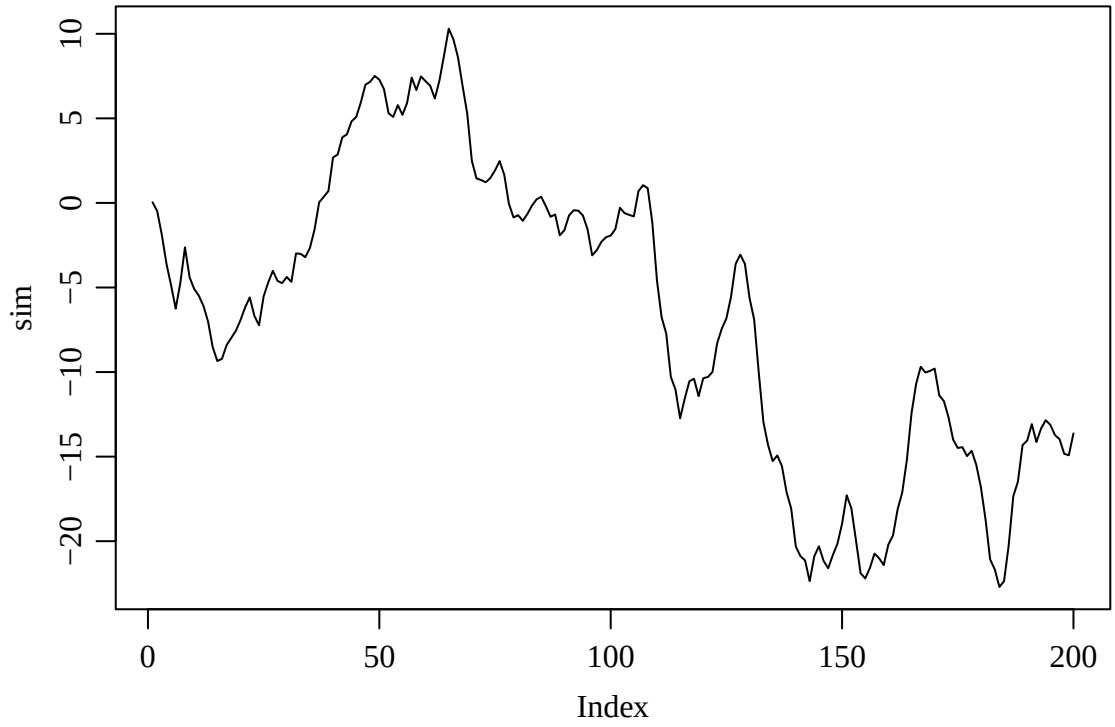
```r
par(mfrow = c(1, 1))
n <- 200; phi <- 1.5
set.seed(128)
z <- rnorm(n)
eta <- numeric(n)
eta[1] <- z[1]
for (i in 2:n) eta[i] <- phi * eta[i - 1] + z[i]
par(mfrow = c(1, 1), mar = c(3.5, 3.5, 0.8, 0.6), mgp = c(2, 0.7, 0), family = "serif")
plot(1:n, eta, las = 1, type = "l", ylab = "", xlab = "")
grid()
```



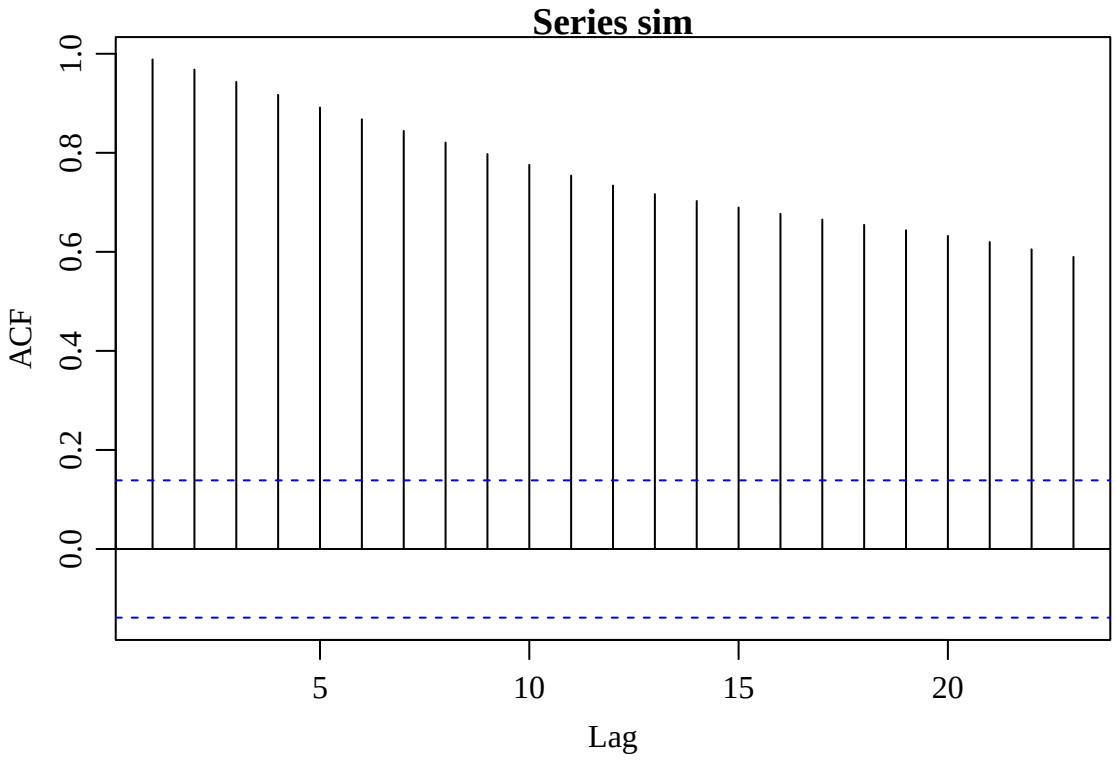

```
acf(eta)
```




```
par(mfrow = c(1, 1), mar = c(3.5, 3.5, 0.8, 0.6), mgp = c(2, 0.7, 0), family = "serif")
sim <- arima.sim(list(order = c(1, 1, 0), ar = 0.5), n = n)[-1]
plot(sim, type = "l")
```




```
acf(sim)
```



A smooth curve fitted to an integrated series

A smooth deterministic curve can follow a realized stochastic trend. That does not establish that the data were generated by a deterministic trend. The red curve below is a descriptive GAM smooth; an ARIMA

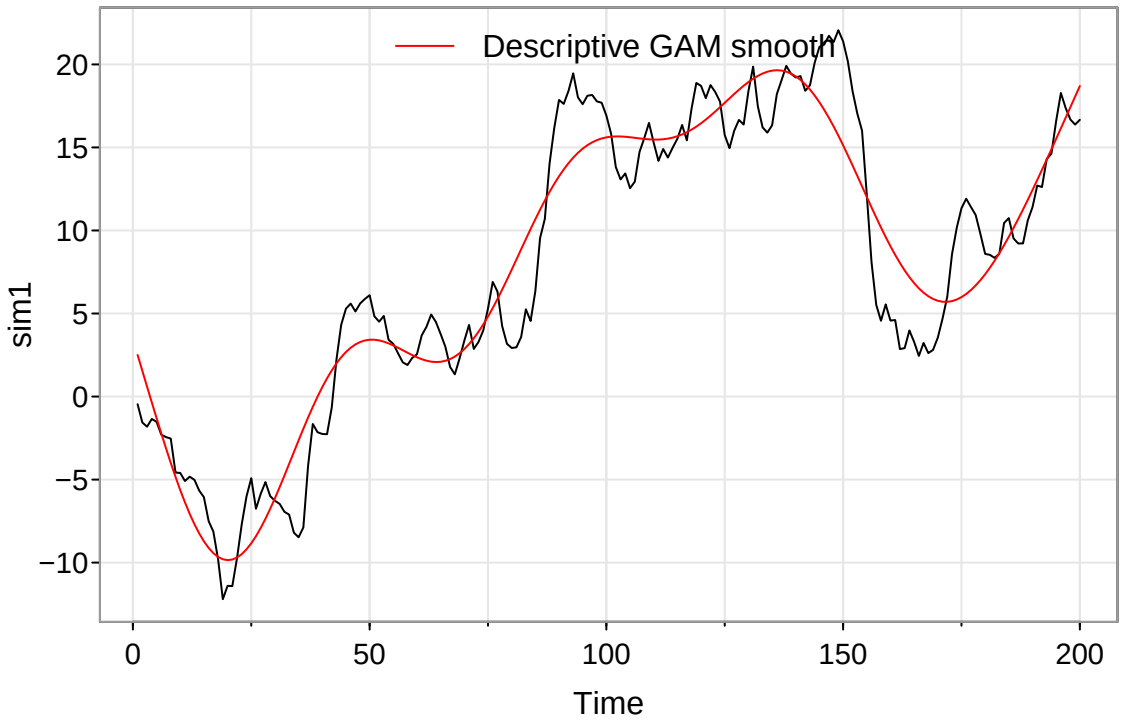
model instead describes the stochastic evolution of the series.

```
set.seed(1235)
n <- 200
t <- 1:n
sim1 <- arima.sim(list(order = c(1, 1, 0), ar = 0.6), n = n)[-1]

par(las = 1, mar = c(3.5, 3.5, 1, 0.5))
tsplot(sim1)
# Fit a descriptive smooth curve.
dFit <- gam(sim1 ~ s(t, k = 10))
lines(t, predict(dFit), col = "red")
# Fit the integrated model separately; it is not a trend curve.
sFit <- arima(sim1, order = c(1, 1, 0))
sFit
```

```
##
## Call:
## arima(x = sim1, order = c(1, 1, 0))
##
## Coefficients:
##          ar1
##       0.4684
## s.e.  0.0624
##
## sigma^2 estimated as 1.139:  log likelihood = -295.42,  aic = 592.84
```

```
legend("top", legend = "Descriptive GAM smooth",
       col = "red", lty = 1, bty = "n")
```



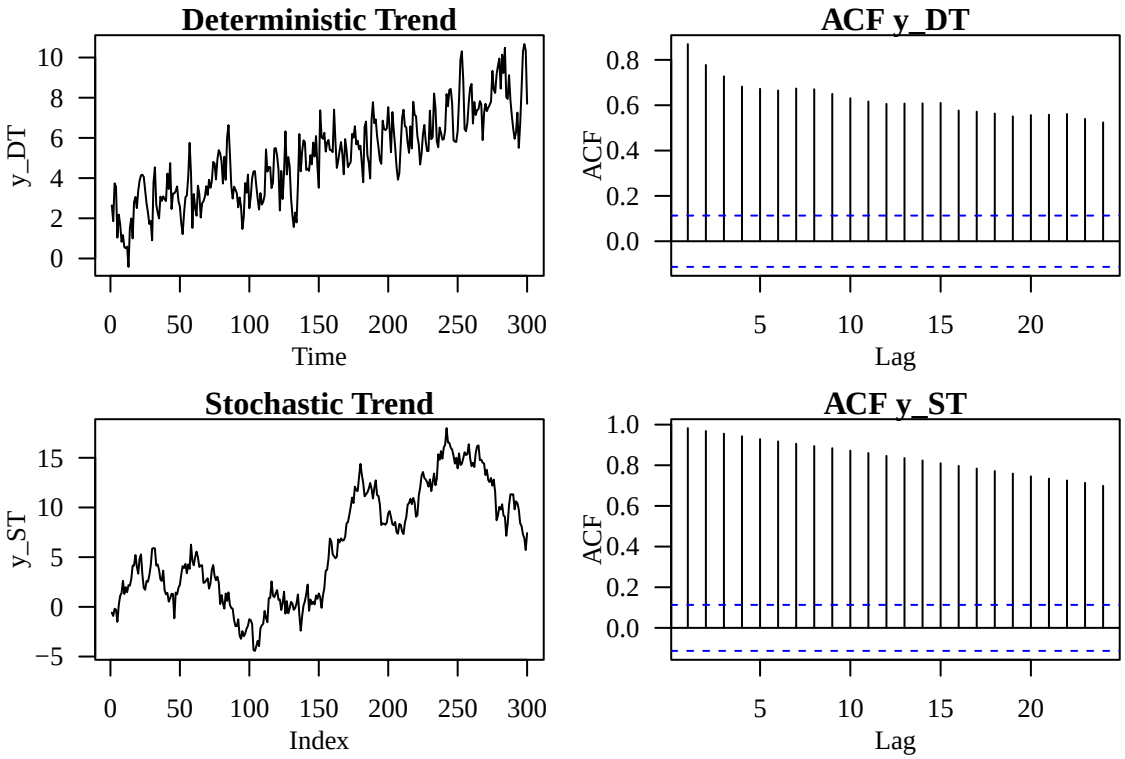
Compare deterministic and stochastic trends

The deterministic-trend example fluctuates around a fixed straight line. The stochastic-trend example is a random walk with drift, so shocks accumulate in its level. Both can show a slowly decaying sample ACF before stationarization.

```
set.seed(123)
n <- 300; t <- 1:n
# Fixed linear trend with stationary AR(1) noise.
u <- arima.sim(list(order = c(1, 0, 0), ar = 0.6), n = n)
y_DT <- 2 + 0.02 * t + u

# Random walk with drift: shocks accumulate.
e <- rnorm(n)
y_ST <- cumsum(0.02 + e)

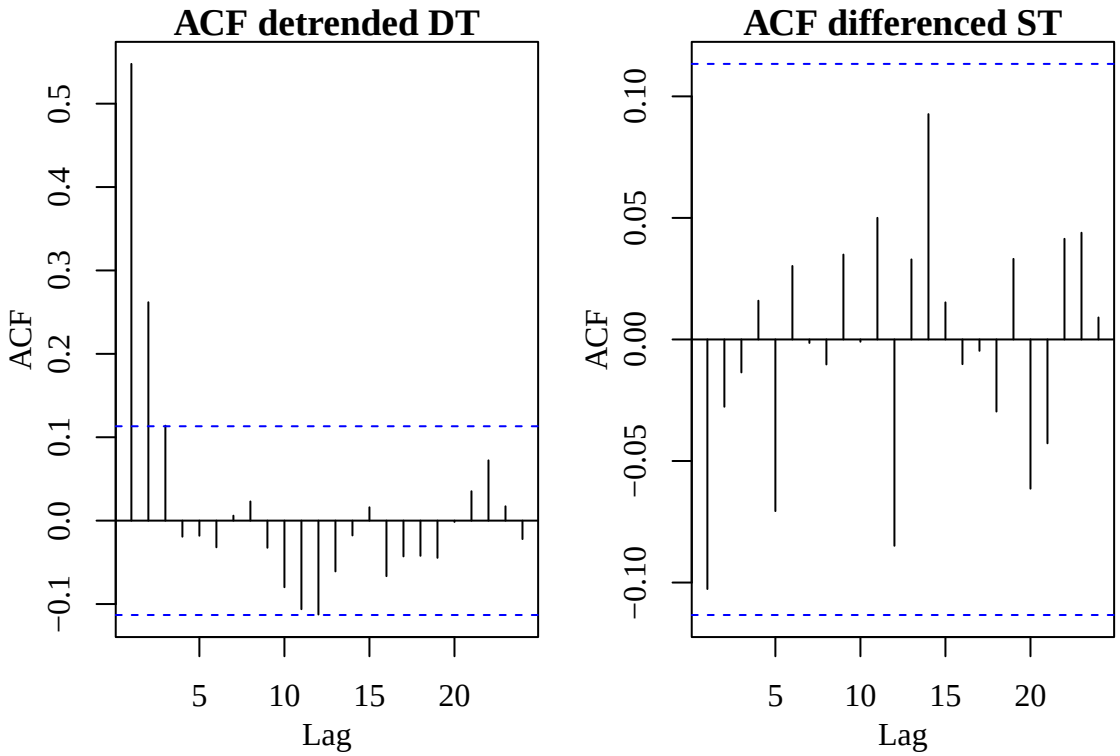
par(mfrow = c(2, 2), las = 1, mar = c(3.5, 3.5, 1, 0.5),
    family = "serif", mgp = c(2, 1, 0))
plot(y_DT, main = "Deterministic Trend")
acf(y_DT, main = "ACF y_DT")
plot(y_ST, type = "l", main = "Stochastic Trend")
acf(y_ST, main = "ACF y_ST")
```



For these known generating mechanisms, remove the deterministic linear trend from the first series and difference the random walk. The detrended series still has AR(1) dependence; the differenced random walk has white-noise changes around its drift.

```
par(mfrow = c(1, 2), mar = c(3.5, 3.5, 1, 0.5), family = "serif", mgp = c(2, 1, 0))
# Match the operation to the generating mechanism.
```

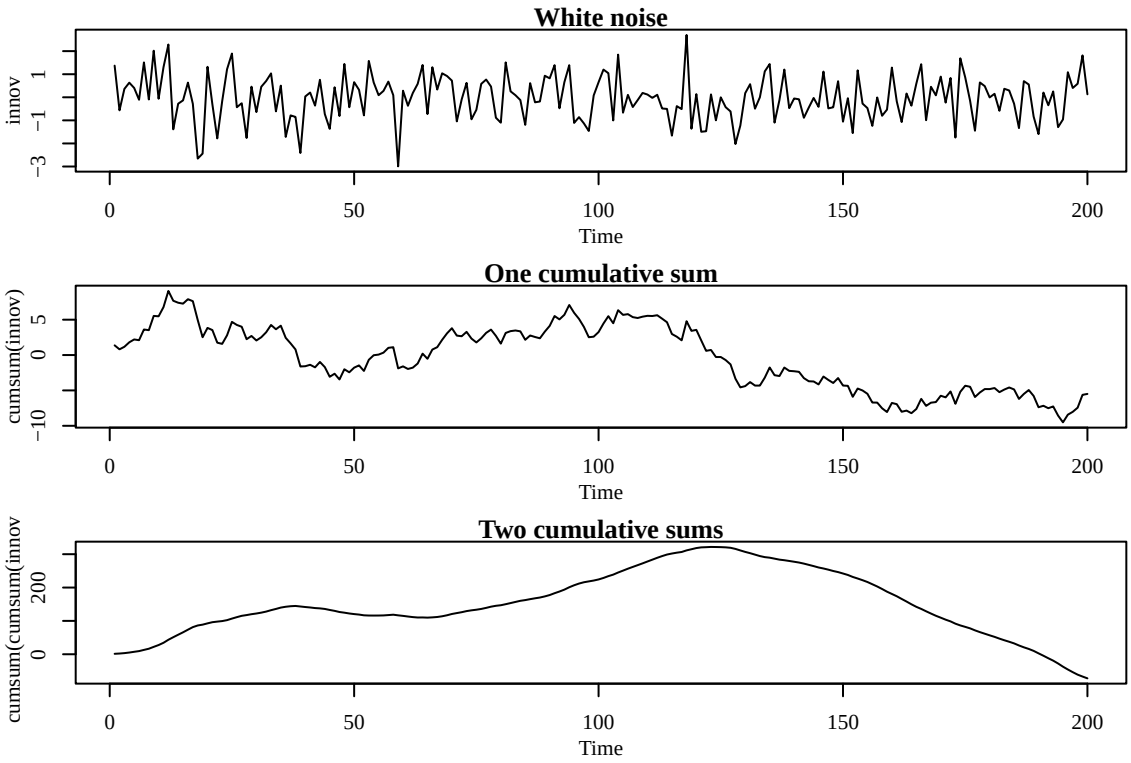
```r
acf(resid(lm(y_DT ~ t)), main = "ACF detrended DT")
acf(diff(y_ST), main = "ACF differenced ST")
```



Integration means accumulating changes

Start with white noise, then accumulate it once and twice. Each cumulative sum adds a level of integration; differencing reverses that operation.

```r
set.seed(42)
innov <- rnorm(200)
par(mfrow = c(3, 1), mar = c(3.5, 3.5, 1, 0.5), family = "serif", mgp = c(2, 1, 0))
ts.plot(innov, main = "White noise")
ts.plot(cumsum(innov), main = "One cumulative sum")
ts.plot(cumsum(cumsum(innov)), main = "Two cumulative sums")
```



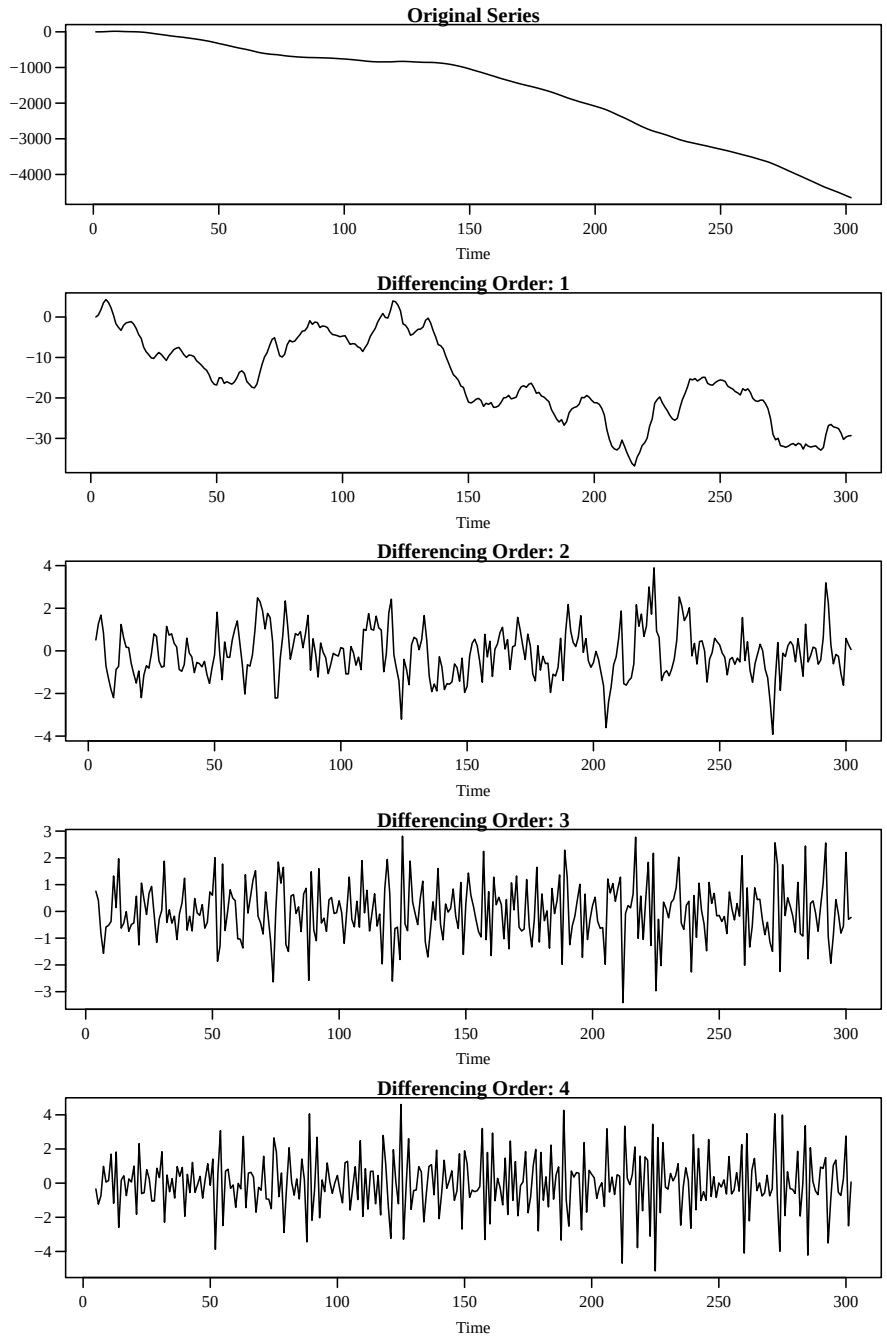
Too little and too much differencing

For this ARIMA(1, 2, 0) simulation, two differences recover a stationary AR(1). Further differencing is unnecessary and changes its dependence structure. Use the smallest justified order, rather than continuing until the plot looks as rough as possible.

```
set.seed(124)
par(mfrow = c(5, 1), mar = c(3.5, 3.5, 1, 0.5), family = "serif",
    mgp = c(2, 0.7, 0), las = 1)
sim <- arima.sim(list(order = c(1, 2, 0), ar = 0.6), n = n)

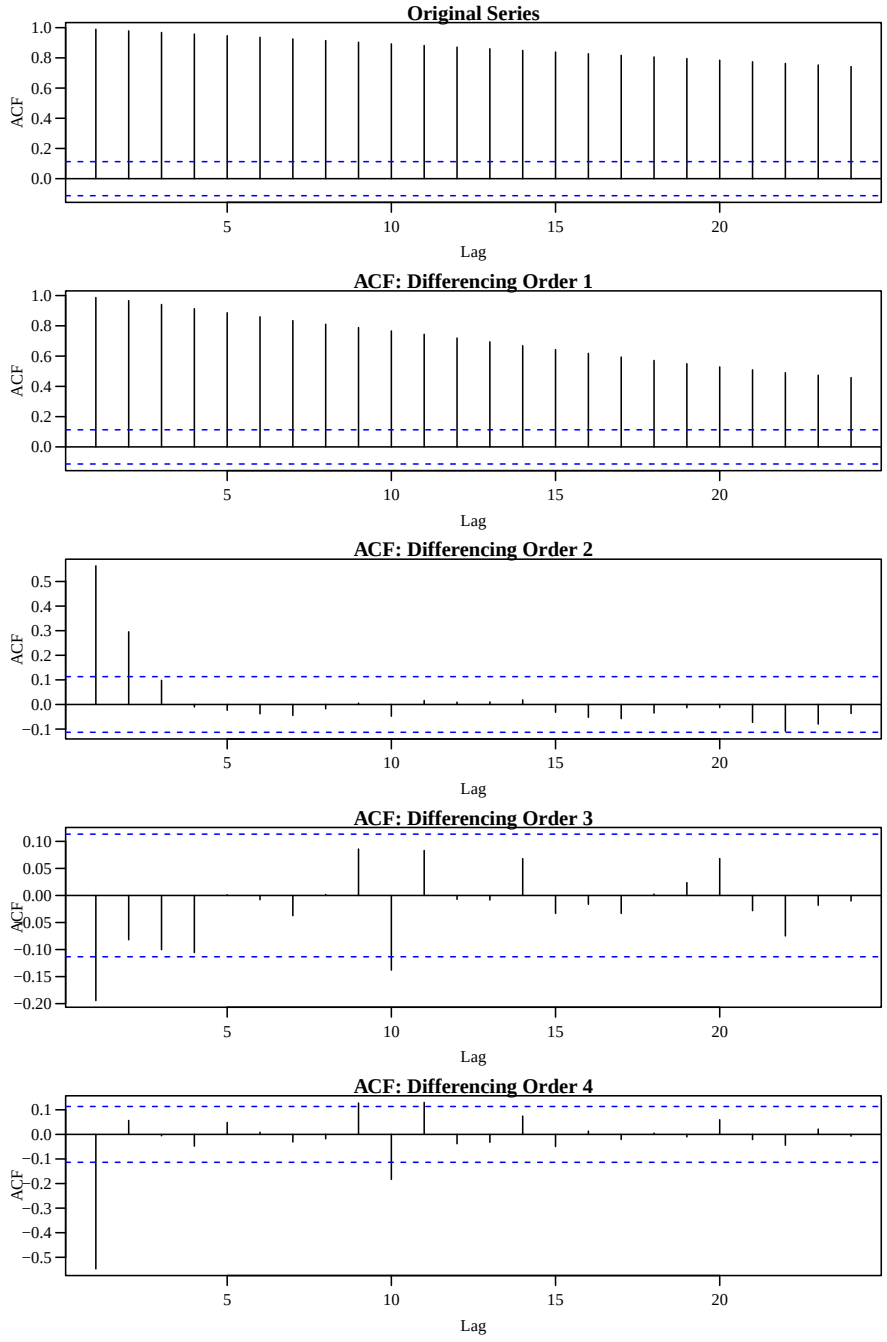
ts.plot(sim, main = "Original Series", ylab = "")

for (d in 1:4) {
  ts.plot(diff(sim, differences = d), ylab = "",
          main = paste("Differencing Order:", d))
}
```




```r
acf(sim, main = "Original Series")

for (d in 1:4) {
  acf(diff(sim, differences = d),
      main = paste("ACF: Differencing Order", d))
}
```




```
par(mfrow = c(1, 1))
```

Forecast changes, then reconstruct levels

For ARIMA(1, 1, 0), let $Y_t = \Delta X_t$ and $Y_t = \phi Y_{t-1} + Z_t$, with zero-mean differences. Then

$$\widehat{Y}_{n+h|n} = \phi^h Y_n, \qquad \widehat{X}_{n+h|n} = X_n + \sum_{j=1}^{h} \widehat{Y}_{n+j|n}.$$

The predicted changes approach zero, but the level forecast retains the current level.

```r
phi <- 0.5
current.level <- 100
last.change <- 4
h <- 1:4

predicted.change <- phi^h * last.change
predicted.level <- current.level + cumsum(predicted.change)
knitr::kable(data.frame(Horizon = h, Change = predicted.change,
                        Level = predicted.level))
```

```
## Warning: 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")

## Warning: 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")
```

Horizon	Change	Level
1	2.00	102.00
2	1.00	103.00
3	0.50	103.50
4	0.25	103.75

For a random walk without drift, the level forecast is constant while its forecast-error variance grows as $h\sigma^2$.

Seasonal dependence and SARIMA

Fixed seasonal patterns versus stochastic seasonal dependence

The first series has a fixed harmonic seasonal mean plus white noise. The second has stationary seasonal AR dependence together with nonseasonal ARMA terms. Both can show seasonal correlation, but their generating mechanisms differ. Here neither model requires seasonal differencing.

```r
n <- 120
t <- 1:n
# Deterministic seasonality
season_d <- 2 * cos(2 * pi * (t / 12)) + 1 * cos(2 * pi * (t / 6))
+ 0.5 * cos(2 * pi * (t / 3))
```

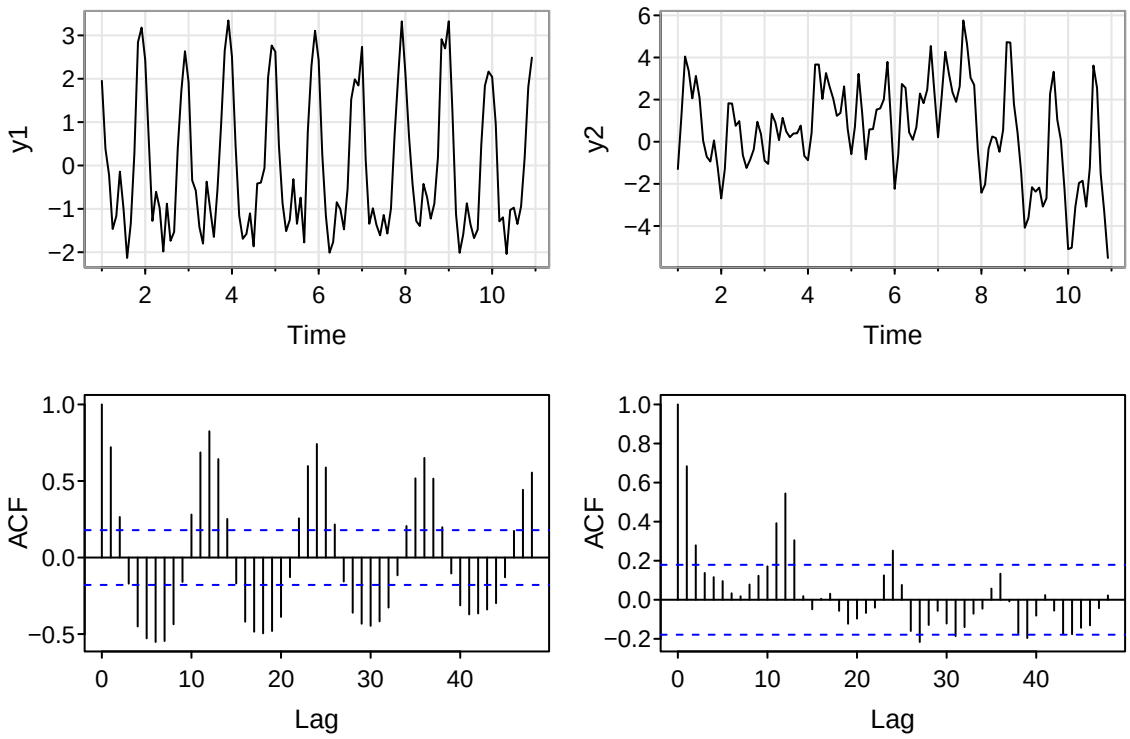
```
##   [1] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
##  [13] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
##  [25] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
##  [37] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
##  [49] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
##  [61] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
##  [73] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
##  [85] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
##  [97] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
## [109] -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50 -0.25 -0.25  0.50
```

```r
set.seed(123)
y1 = season_d + rnorm(n, sd = 0.5)
# Convert to a time series with monthly frequency
y1 <- ts(y1, frequency = 12, start = 1)

par(las = 1, mfrow = c(2, 2))
tsplot(y1)

set.seed(12)
# Stochastic seasonality
# Use a simulator that explicitly supports seasonal coefficients.
y2 <- astsa::sarima.sim(ar = 0.8, ma = 0.95, sar = 0.9,
                        S = 12, n = n, sd = 0.75)
# Convert to a time series with monthly frequency
y2 <- ts(y2, frequency = 12, start = 1)
tsplot(y2)

stats::acf(as.numeric(y1), lag.max = 48, main = "Fixed seasonal mean")
stats::acf(as.numeric(y2), lag.max = 48, main = "Seasonal ARMA dependence")
```

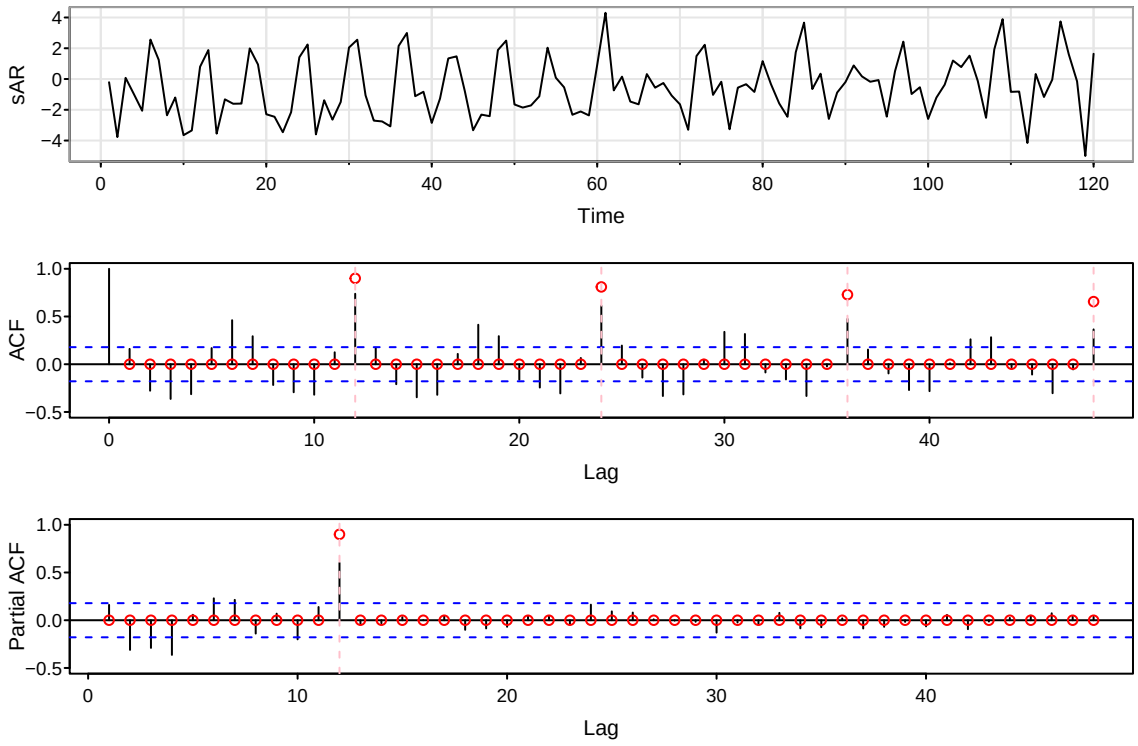


Seasonal AR: dependence on last year's observation

For $Y_t = 0.9Y_{t-12} + Z_t$, dependence occurs at multiples of lag 12. Compare the sample ACF/PACF with their theoretical values (red points). A finite sample will not reproduce the theoretical pattern exactly.

```r
n <- 120; Phi = 0.9
set.seed(1234)
sAR = sarima.sim(sar = Phi, S = 12, n = n)
```

```r
sAR <- ts(sAR, frequency = 1, start = 1)
par(las = 1, mar = c(3.5, 3.5, 1, 0.5), mgp = c(2.5, 1, 0), mfrow = c(3, 1))
tsplot(sAR, xlab = "Time")
stats::acf(sAR, lag.max = 48, ylim = c(-0.5, 1))
trueACF <- ARMAacf(ar = c(rep(0, 11), Phi), lag.max = 48)
points(1:48, trueACF[2:49], col = "red")
abline(v = 12 * (1:4), col = "pink", lty = 2)
stats::pacf(sAR, lag.max = 48, ylim = c(-0.5, 1))
truePACF <- ARMAacf(ar = c(rep(0, 11), Phi), lag.max = 48, pacf = TRUE)
points(1:48, truePACF, col = "red")
abline(v = 12, col = "pink", lty = 2)
```

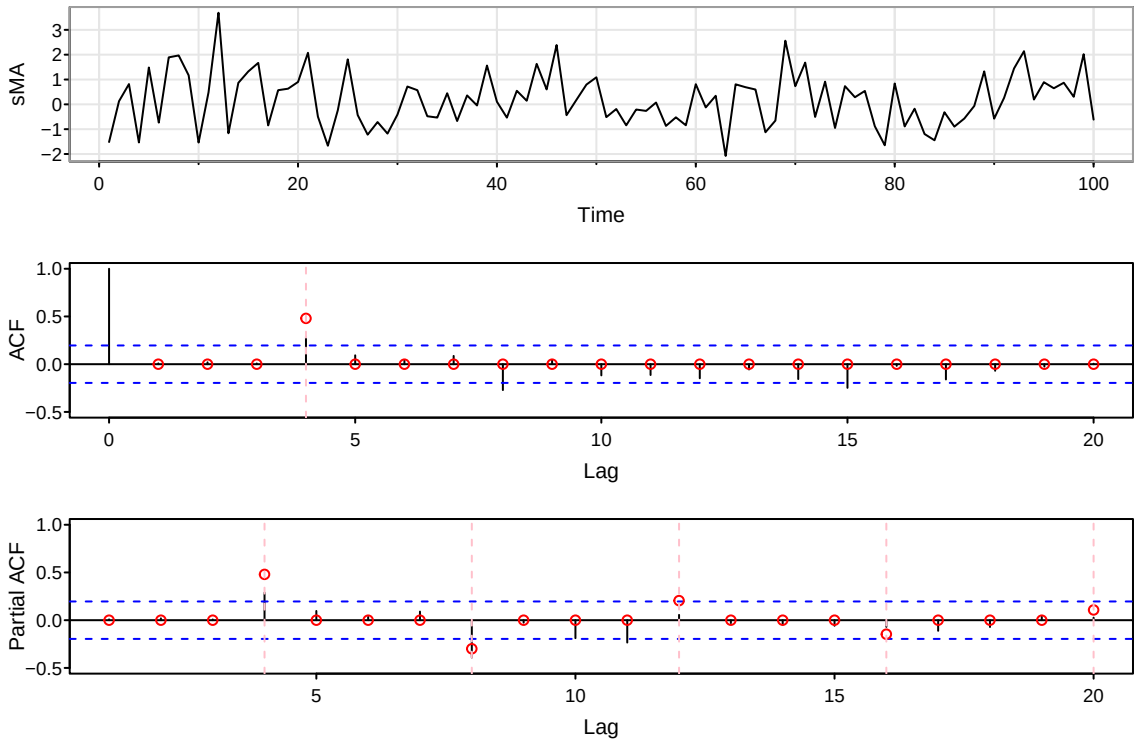


Seasonal MA: dependence on a past seasonal shock

For $Y_t = Z_t + 0.75Z_{t-4}$, the theoretical ACF cuts off after the seasonal MA lag, whereas the PACF continues at seasonal multiples. Red points show the theoretical values.

```r
n <- 100; Theta = 0.75
set.seed(1234)
sMA = sarima.sim(sma = Theta, S = 4, n = n)
sMA <- ts(sMA, frequency = 1, start = 1)
par(las = 1, mar = c(3.5, 3.5, 1, 0.5), mgp = c(2.5, 1, 0), mfrow = c(3, 1))
tsplot(sMA, xlab = "Time")
stats::acf(sMA, lag.max = 20, ylim = c(-0.5, 1))
trueACF <- ARMAacf(ma = c(rep(0, 3), Theta), lag.max = 20)
points(1:20, trueACF[2:21], col = "red")
abline(v = 4, col = "pink", lty = 2)
stats::pacf(sMA, lag.max = 20, ylim = c(-0.5, 1))
```

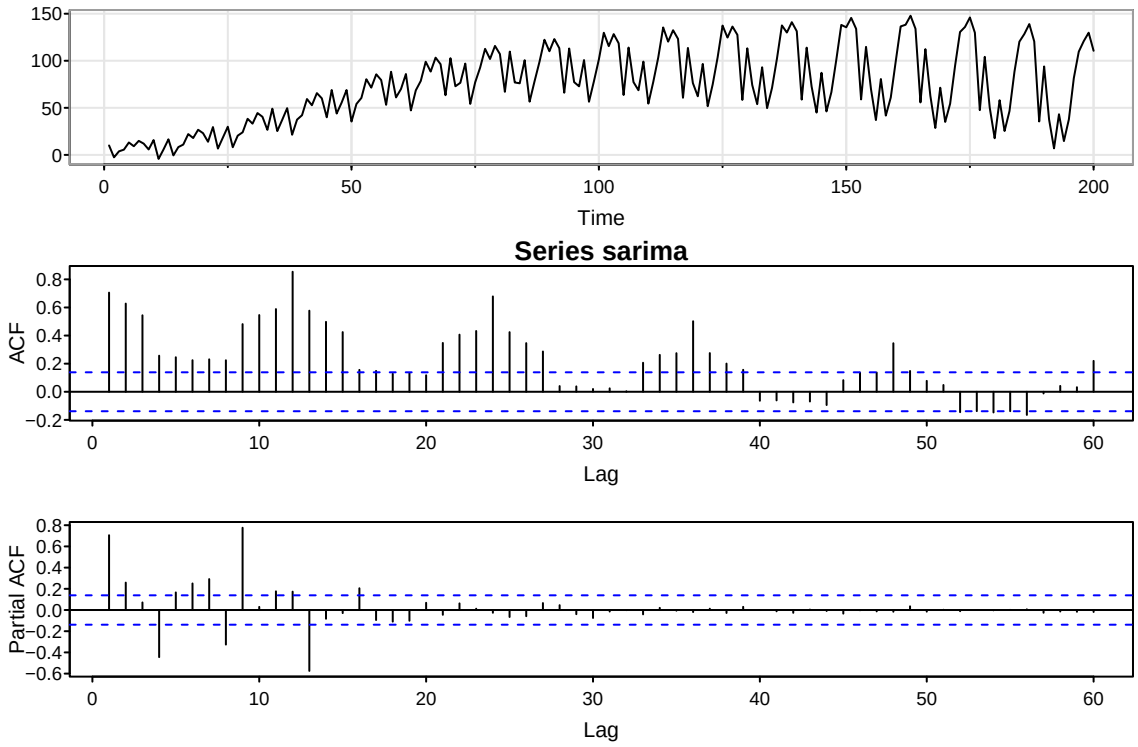
```r
truePACF <- ARMAacf(ma = c(rep(0, 3), Theta), lag.max = 20, pacf = TRUE)
points(1:20, truePACF, col = "red")
abline(v = 4 * (1:5), col = "pink", lty = 2)
```



Combine ordinary and seasonal integration

This simulation combines ordinary and seasonal differences with AR and MA components. The level plot and its ACF/PACF illustrate the persistent structure before differencing; ARMA order identification should use the appropriately differenced series.

```r
par(las = 1, mar = c(3.5, 3.5, 1, 0.5), mgp = c(2.5, 1, 0), mfrow = c(3, 1))
set.seed(123)
sarima <- sarima.sim(d = 1, ar = -.25, sar = .9, D = 1, sma = 0.75, S = 12, n = 200)
sarima <- ts(sarima, frequency = 1, start = 1)
tsplot(sarima, ylab = "")
acf(sarima, lag.max = 60)
pacf(sarima, lag.max = 60)
```



Monthly International Airline Passenger Data

Load the data and inspect its time index

We retain the original `TSA::airpass` example. Check the stored dates rather than assigning dates from another airline dataset. The last 12 observations will be held out for forecast evaluation.

```
data(airpass)
start(airpass)
```

```
## [1] 1960    1
```

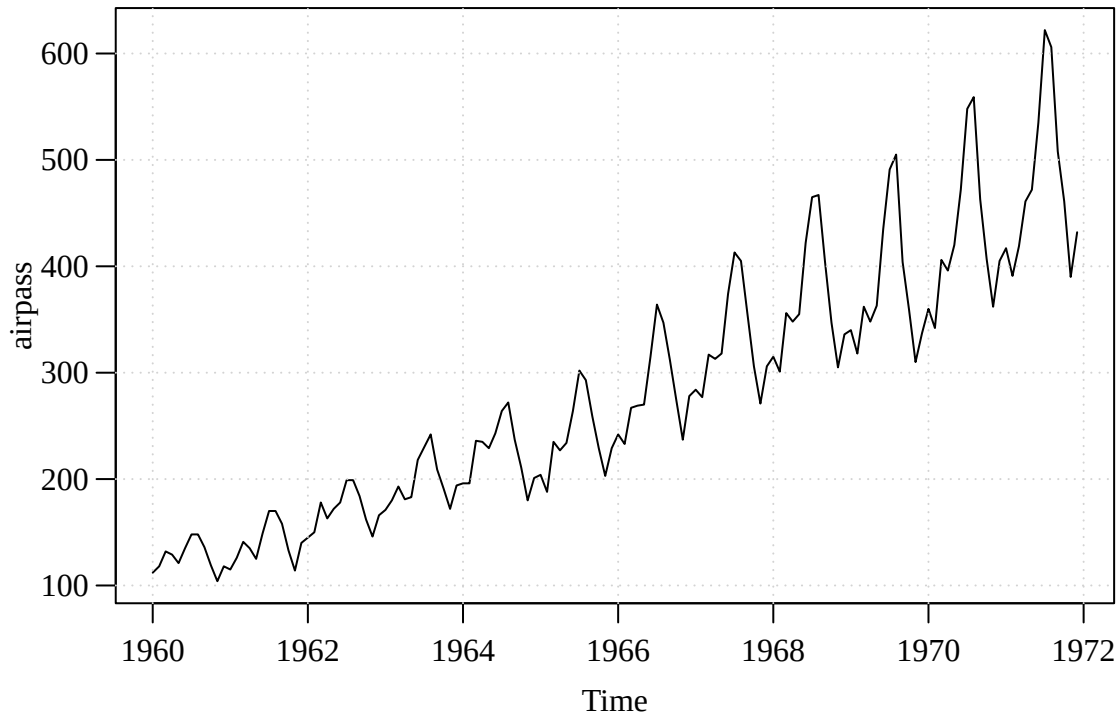
```
end(airpass)
```

```
## [1] 1971   12
```

```
frequency(airpass)
```

```
## [1] 12
```

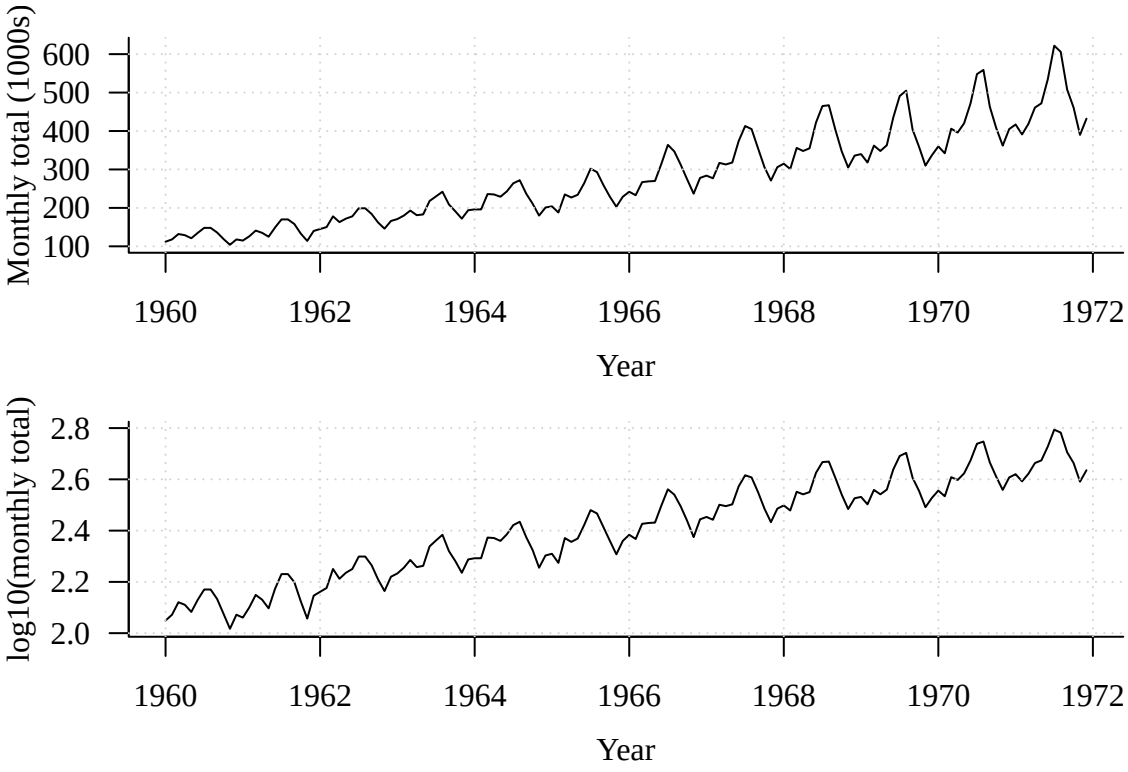
```
par(mfrow = c(1, 1), mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 0.7, 0),
    family = "serif", las = 1)
plot(airpass)
grid()
```



Stabilize variability with a log transformation

Seasonal fluctuations grow with the passenger level. Compare the raw series with its base-10 logarithm. The transformation makes variability more comparable, but does not by itself remove trend or seasonal dependence.

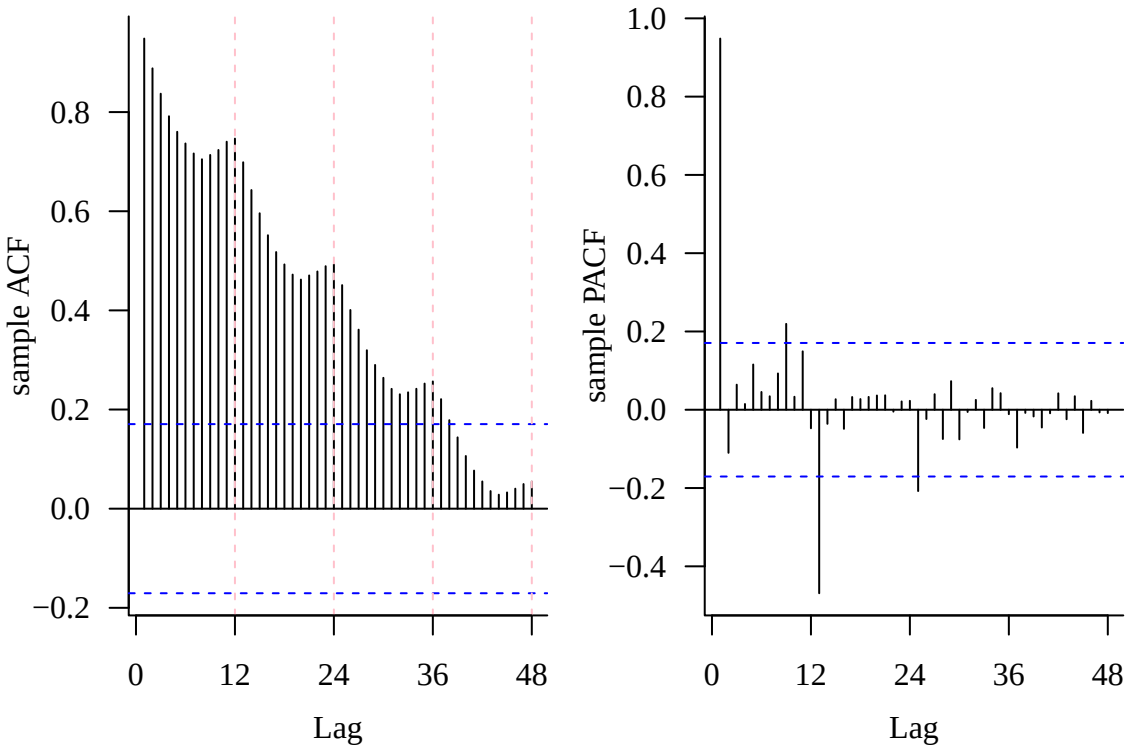
```
par(bty = "L", mar = c(3.6, 3.5, 0.8, 0.6), mgp = c(2.4, 1, 0),
    las = 1, mfrow = c(2, 1), family = "serif")
## plot the time series.
plot(airpass, xlab = "Year", ylab = "Monthly total (1000s)")
grid()
## take a log (to the base 10) of the air passenger data.
log.airpass <- log10(airpass)
plot(log.airpass, type = "l", xlab = "Year", ylab = "log10(monthly total)")
grid()
```



Define the training period and inspect dependence

Use the first 132 observations for model development and reserve the final 12 for evaluation. All differencing choices and model diagnostics below use only the training period. The vectors passed to ACF/PACF use observation lags, so lag 12 represents one year.

```
yr <- time(airpass)
log.shortair <- log.airpass[1:132]
shortyears <- yr[1:132]
par(bty = "L", mar = c(3.6, 3.5, 0.8, 0.6), mgp = c(2.4, 1, 0), las = 1,
    mfrow = c(1, 2), family = "serif")
acf(log.shortair, ylab = "sample ACF", main = "", lag.max = 48, xaxt = "n")
abline(v = 12 * (1:4), col = "pink", lty = 2)
axis(side = 1, at = seq(0, 48, 12))
pacf(log.shortair, ylab = "sample PACF", main = "", lag.max = 48, xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))
```



Compare ordinary and seasonal differencing

Compare $(d, D) = (1, 0), (0, 1), (1, 1)$. Ordinary differencing compares consecutive months; seasonal differencing compares the same month in adjacent years. Both operations are not automatically necessary.

```
## take the differences Y_t = (1-B) X_t
diff.1.0 <- diff(log.shortair)
## take the seasonal differences Y_t =(1-B^(12)) X_t
diff.0.1 <- diff(log.shortair, lag = 12, differences = 1)
## take the differences Y_t = (1-B^(12)) (1-B) X_t
diff.1.1 <- diff(diff(log.shortair, lag = 12, differences = 1))
```

Assess the three differencing choices

Read each row from left to right: the differenced series, its ACF, and its PACF. Look for a stable level and remaining short-term or seasonal dependence. Strong negative correlations can be a warning of unnecessary differencing.

```
par(bty = "L", mar = c(3.6, 3.5, 1, 0.6), mgp = c(2.4, 1, 0), las = 1)
layout.matrix <- matrix(c(1, 1, 2, 3, 4, 4, 5, 6, 7, 7, 8, 9), nrow = 3,
                        ncol = 4, byrow = TRUE)
layout(mat = layout.matrix)
plot(shortyears[-1], diff.1.0, xlab = "", ylab = "d=1, D=0",
     type = "l", ylim = c(-0.1, 0.1), xlim = range(shortyears))

stats::acf(diff.1.0, lag.max = 48, ylab = "", xlab = "", main = "",
           ylim = c(-0.6, 1), xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))
```

```r
mtext("Sample ACF", side = 3, line = 0, cex = 0.8)

stats::pacf(diff.1.0, lag.max = 48, ylab = "", xlab = "", main = "",
            ylim = c(-0.6, 1), xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))
mtext("Sample PACF", side = 3, line = 0, cex = 0.8)

plot(shortyears[-c(1:12)], diff.0.1, xlab = "", ylab = "d=0, D=1",
     type = "l", ylim = c(-0.1, 0.1), xlim = range(shortyears))

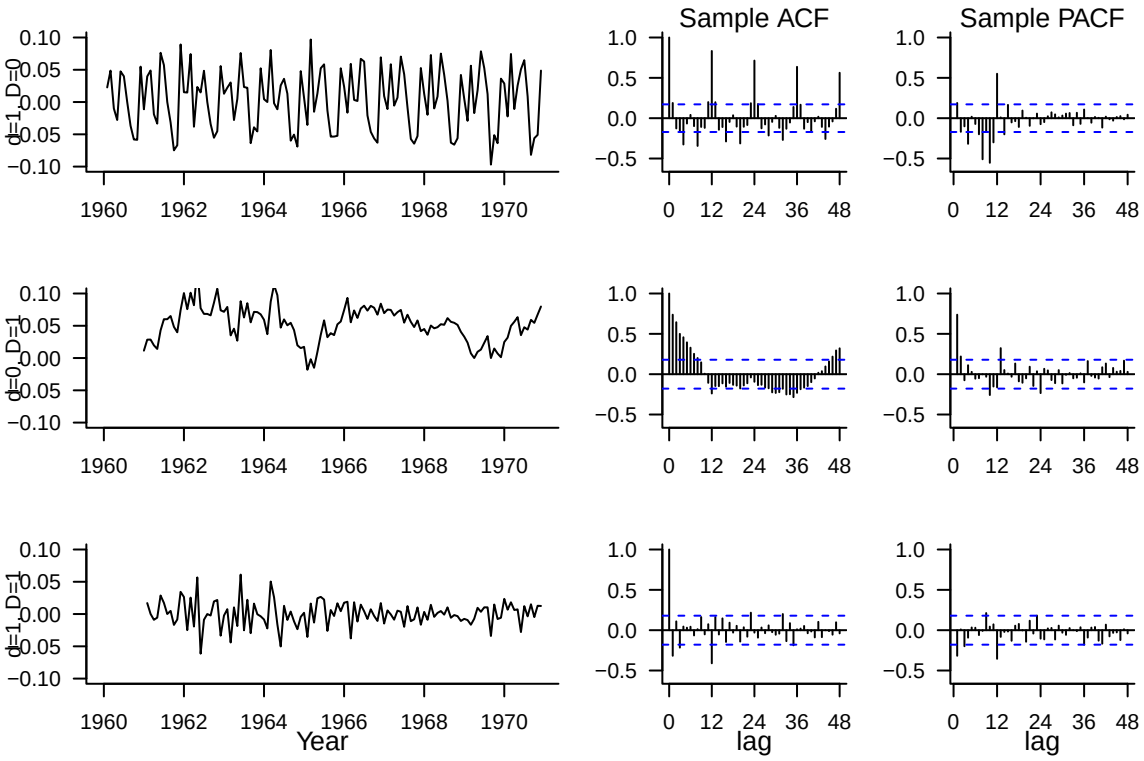
stats::acf(diff.0.1, lag.max = 48, ylab = "", xlab = "", main = "",
           ylim = c(-0.6, 1), xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))

stats::pacf(diff.0.1, lag.max = 48, ylab = "", xlab = "", main = "",
            ylim = c(-0.6, 1), xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))

plot(shortyears[-c(1:13)], diff.1.1, xlab = "", ylab = "d=1, D=1",
     type = "l", ylim = c(-0.1, 0.1), xlim = range(shortyears))
mtext("Year", side = 1, line = 1.8, cex = 0.8)

stats::acf(diff.1.1, lag.max = 48, ylab = "", xlab = "", main = "",
           ylim = c(-0.6, 1), xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))
mtext("lag", side = 1, line = 1.8, cex = 0.8)

stats::pacf(diff.1.1, lag.max = 48, ylab = "", xlab = "", main = "",
            ylim = c(-0.6, 1), xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))
mtext("lag", side = 1, line = 1.8, cex = 0.8)
```

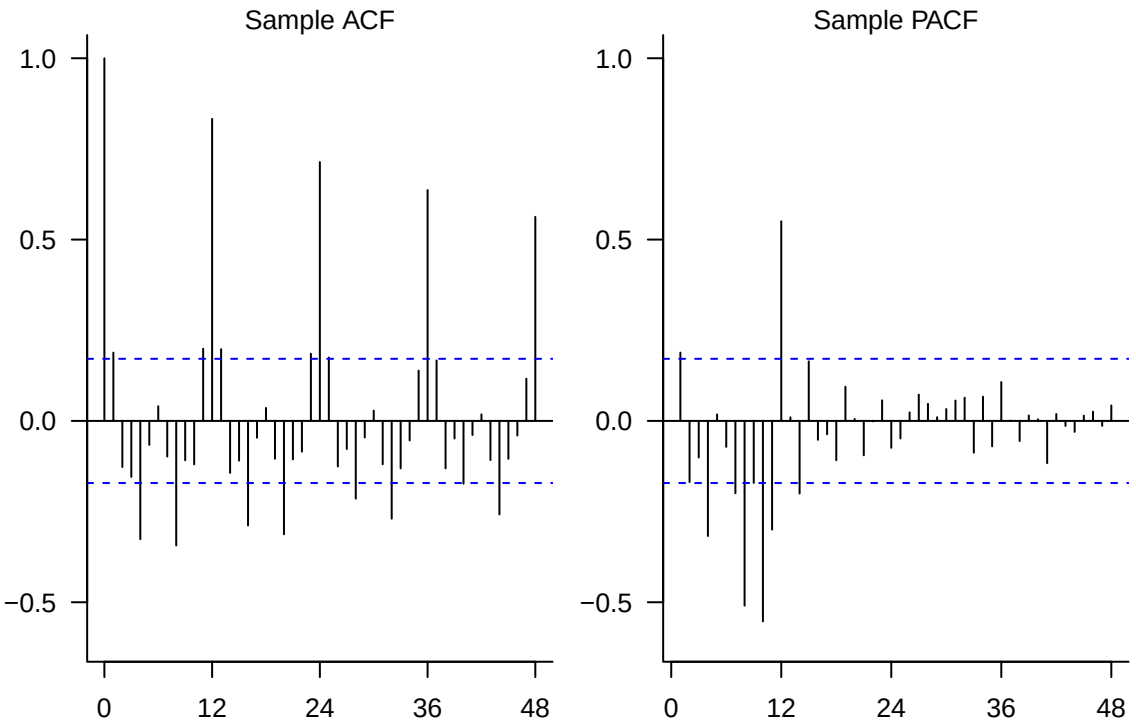


Develop candidates using one ordinary difference

Continue with $d = 1, D = 0$ as in the lecture. Remaining annual dependence is modeled by a seasonal AR term rather than a seasonal difference. This is a candidate modeling choice, to be assessed through residuals and forecasts.

```
par(mfrow = c(1, 2), cex = 0.8, bty = "L", mar = c(3.6, 3, 1, 0.6), mgp = c(2.4, 1, 0), las = 1)
stats::acf(diff.1.0, lag.max = 48, ylab = "", xlab = "", main = "",
           ylim = c(-0.6, 1), xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))
mtext("Sample ACF", side = 3, cex = 0.8)

stats::pacf(diff.1.0, lag.max = 48, ylab = "", xlab = "", main = "",
            ylim = c(-0.6, 1), xaxt = "n")
axis(side = 1, at = seq(0, 48, 12))
mtext("Sample PACF", side = 3, cex = 0.8)
```



A helper for residual diagnostics

The following helper, adapted from Peter Craigmile, displays the residual series, Q-Q plot, ACF, and PACF, and returns a Ljung-Box test. The test assesses residual autocorrelation at a collection of lags; a large p-value does not establish independence or normality.

```
plot.residuals <- function (x, y = NULL, lag.max = NULL, mean.line = TRUE,
                 acf.ylim = c(-0.25, 1), mfrow = c(2, 2),
                             lags = NULL, fitdf = 0, ...){
  old.par <- par(no.readonly = TRUE)
  on.exit(par(old.par))
  if (!is.null(mfrow))
    par(mfrow = mfrow)
  if (is.null(y)){
    y <- x
    x <- seq(length(y))
  } else {
    x <- as.numeric(x)
    y <- as.numeric(y)
  }

  if (is.null(lag.max)) {
    lag.max <-  floor(10 * log10(length(x)))
  }
  plot(x, y, type = "l", ...)
  if (mean.line) abline(h = 0, lty = 2)
  qqnorm(y, main = "", las = 1); qqline(y)
  if (is.null(lags)) {
    stats::acf(y, main = "", lag.max = lag.max, xlim = c(0, lag.max), ylim = acf.ylim,
        ylab = "sample ACF", las = 1)
```

```
    stats::pacf(y, main = "", lag.max = lag.max, xlim = c(0, lag.max), ylim = acf.ylim,
         ylab = "sample PACF", las = 1)
  }
  else {
    stats::acf(y, main = "", lag.max = lag.max, xlim = c(0, lag.max), ylim = acf.ylim,
        ylab = "sample ACF", xaxt = "n", las = 1)
    axis(side = 1, at = lags)

    stats::pacf(y, main = "", lag.max = lag.max, xlim = c(0, lag.max), ylim = acf.ylim,
         ylab = "sample PACF", xaxt = "n", las = 1)
    axis(side = 1, at = lags)
  }
  Box.test(y, lag = lag.max, type = "Ljung-Box", fitdf = fitdf)
}
```

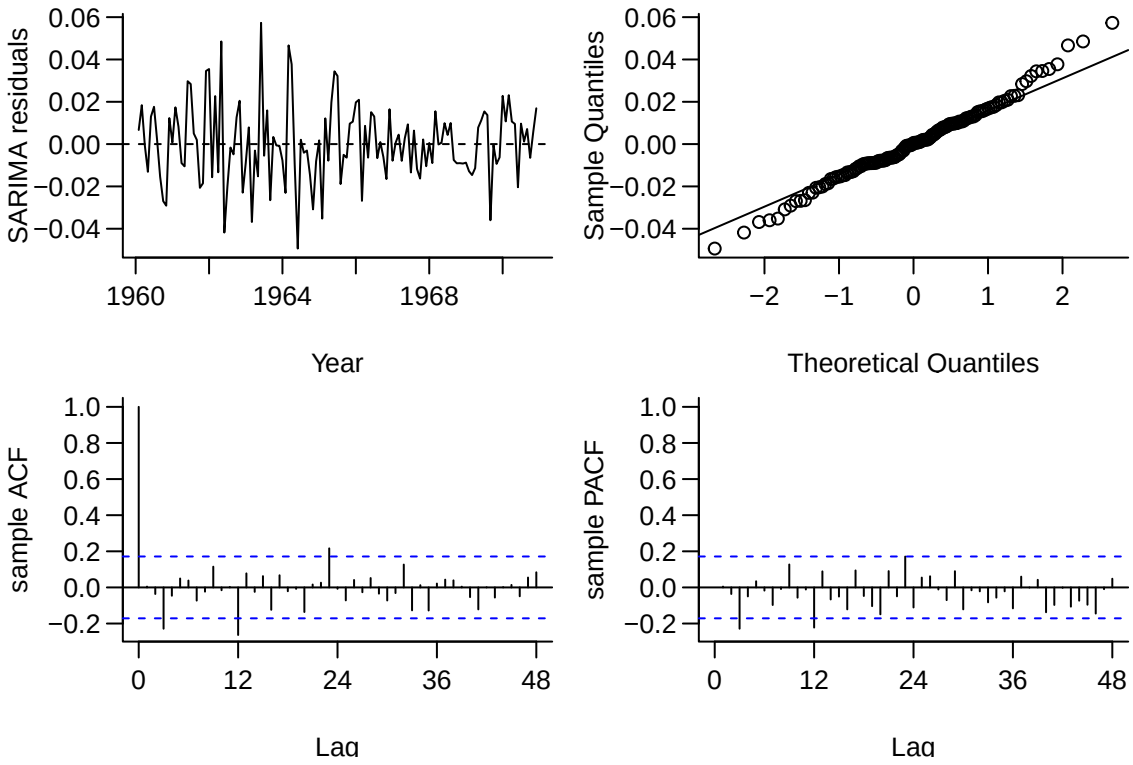
Candidate 1: nonseasonal and seasonal AR terms

Fit AR(1) and seasonal AR(1) terms to the first differences. This corresponds to SARIMA$(1,1,0)\times(1,0,0)_{12}$ for the log levels. Here the fitted intercept is the mean of the stationary differences, not the mean log level. Subtract two fitted AR coefficients in the Ljung-Box degrees of freedom.

```
(fit1 <- arima(diff.1.0, order = c(1, 0, 0), seasonal = list(order = c(1, 0, 0),
                                                             period = 12)))
```

```
##
## Call:
## arima(x = diff.1.0, order = c(1, 0, 0), seasonal = list(order = c(1, 0, 0),
##     period = 12))
##
## Coefficients:
##           ar1    sar1  intercept
##       -0.2667  0.9291     0.0039
## s.e.   0.0865  0.0235     0.0096
##
## sigma^2 estimated as 0.0003298:  log likelihood = 327.27,  aic = -648.54
```

```
par(mfrow = c(2, 2), cex = 0.8, bty = "L", mar = c(3.6, 4, 0.8, 0.6),
    mgp = c(2.8, 0.7, 0), las = 1)
plot.residuals(shortyears[-1], resid(fit1), lag.max = 48,
               ylab = "SARIMA residuals", xlab = "Year",
               lags = seq(0, 48, 12), fitdf = 2)
```




```
##
##  Box-Ljung test
##
## data:  y
## X-squared = 55.372, df = 46, p-value = 0.162
```

Candidate 2: remove the nonseasonal AR term

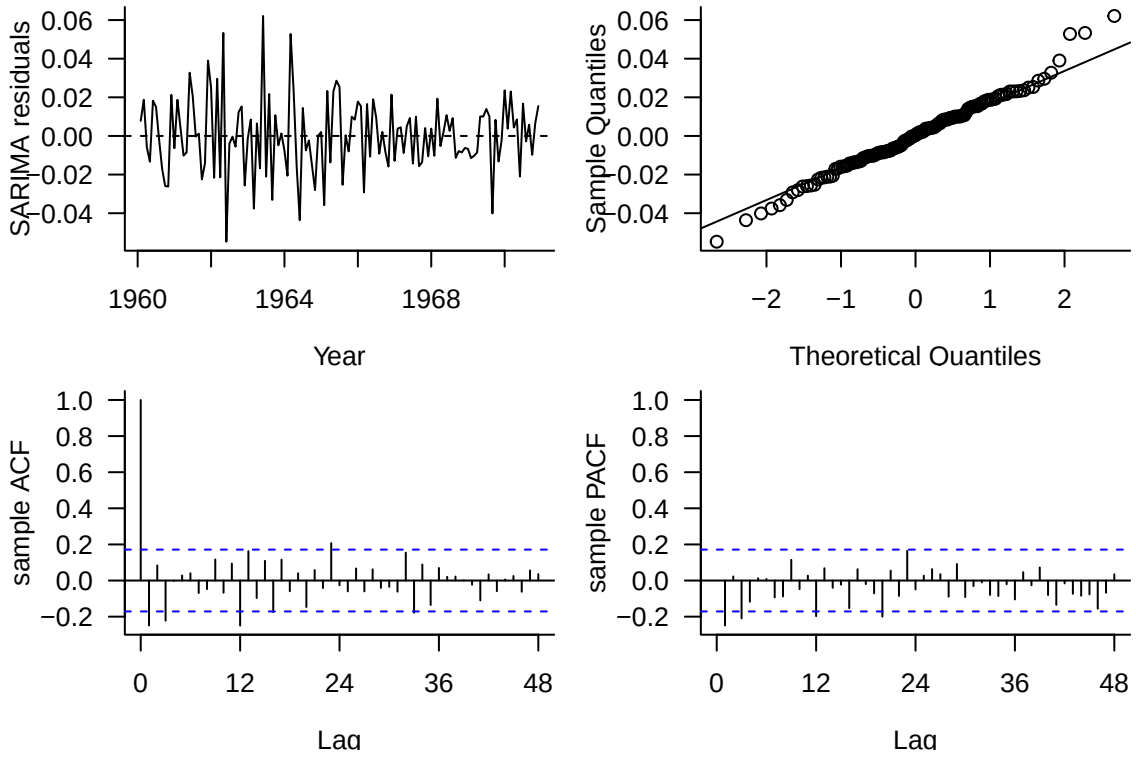
Fit the simpler model to the same differences. Compare its information criterion and residual dependence with Candidate 1. The Ljung-Box adjustment now subtracts one fitted seasonal AR coefficient.

```
(fit2 <- arima(diff.1.0, seasonal = list(order = c(1, 0, 0), period = 12)))
```

```
##
## Call:
## arima(x = diff.1.0, seasonal = list(order = c(1, 0, 0), period = 12))
##
## Coefficients:
##         sar1  intercept
##       0.9081     0.0040
## s.e.  0.0278     0.0108
##
## sigma^2 estimated as 0.0003616:  log likelihood = 322.75,  aic = -641.51
```

```
par(mfrow = c(2, 2), cex = 0.8, bty = "L", mar = c(3.6, 4, 0.8, 0.6),
    mgp = c(2.8, 1, 0), las = 1)
plot.residuals(shortyears[-1], resid(fit2), lag.max = 48,
```

```
              ylab = "SARIMA residuals", xlab = "Year",
              lags = seq(0, 48, 12), fitdf = 1)
```




```
##
##  Box-Ljung test
##
## data:  y
## X-squared = 80.641, df = 47, p-value = 0.001637
```

Fit integrated models for forecasting the held-out year

We now fit the models directly to the training **log levels**, specifying $d = 1$. R then performs differencing and reconstructs level forecasts internally. These calls omit drift, whereas the earlier fits to differences included their mean; consequently, the forecasting fits are not identical parameterizations. We retain this distinction from the original session and make it explicit.

Fit SARIMA$(1,1,0) \times (1,0,0)_{12}$:

```
(fit1 <- arima(log.shortair, order = c(1, 1, 0),
                      seasonal = list(order = c(1, 0, 0), period = 12)))
```

```
##
## Call:
## arima(x = log.shortair, order = c(1, 1, 0), seasonal = list(order = c(1, 0,
##     0), period = 12))
##
## Coefficients:
##           ar1    sar1
```

```
##       -0.2665  0.9298
## s.e.   0.0866  0.0233
##
## sigma^2 estimated as 0.0003299:  log likelihood = 327.19,  aic = -650.38
```

Fit SARIMA$(0,1,0) \times (1,0,0)_{12}$:

```
(fit2 <- arima(log.shortair, order = c(0, 1, 0),
                    seasonal = list(order = c(1, 0, 0), period = 12)))
```

```
##
## Call:
## arima(x = log.shortair, order = c(0, 1, 0), seasonal = list(order = c(1, 0,
##     0), period = 12))
##
## Coefficients:
##         sar1
##       0.9088
## s.e.  0.0276
##
## sigma^2 estimated as 0.0003617:  log likelihood = 322.69,  aic = -643.38
```

Identify the forecast horizon

Use the actual time index of the held-out observations. The forecasting origin is the final training month; all 12 predictions below are made at that origin.

```
fyears <- yr[133:144]
```

Calculate log-level forecasts and prediction intervals

`predict()` returns forecasts of the log levels, not of their differences. Combine the predicted values and standard errors to form approximate Gaussian 95% prediction intervals. These intervals are conditional on the fitted model and do not fully account for parameter or model-selection uncertainty.

```
preds1 <- predict(fit1, n.ahead = 12)
forecast1 <- preds1$pred
flimits1  <- qnorm(0.975) * preds1$se

preds2 <- predict(fit2, n.ahead = 12)
forecast2 <- preds2$pred
flimits2  <- qnorm(0.975) * preds2$se
```

Plot forecasts on the log and original scales

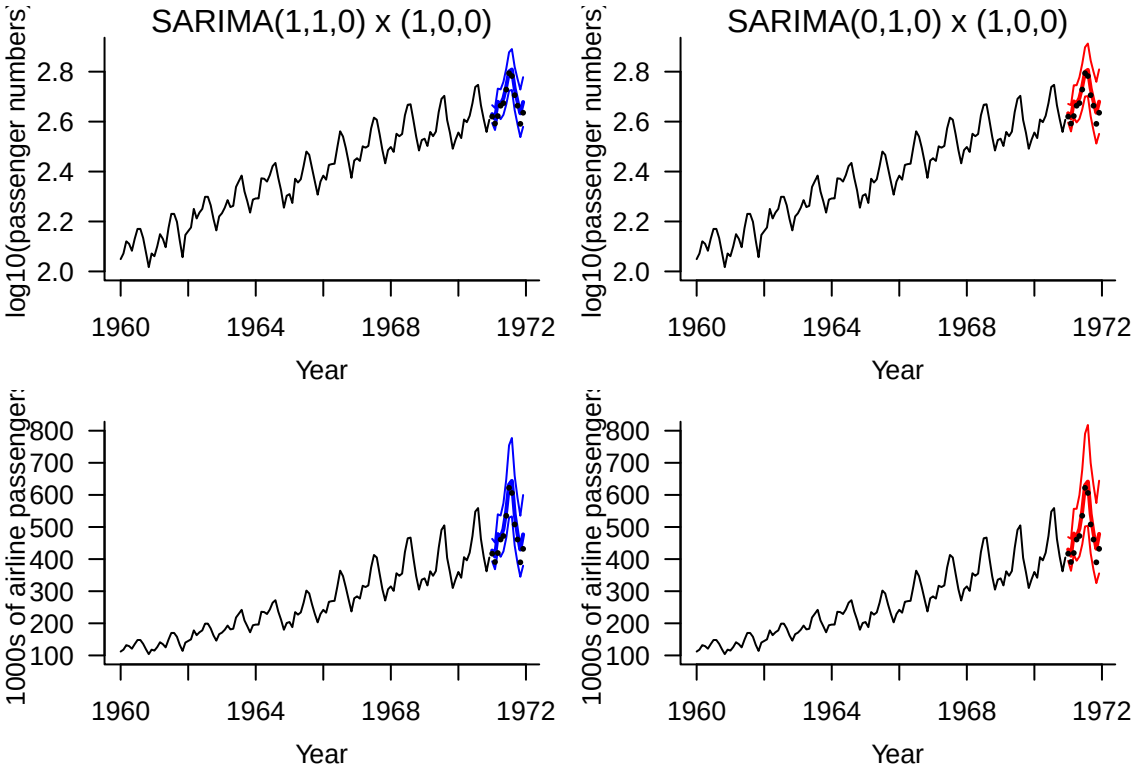
The top panels show log forecasts. The bottom panels exponentiate the point forecasts and interval endpoints. Under a Gaussian predictive distribution on the log scale, these exponentiated point forecasts are **medians**, not means. The intervals become asymmetric on the original scale. Black points show the held-out observations, which were not used in fitting.

```r
par(mfrow = c(2, 2), cex = 0.8, bty = "L", mar = c(3.6, 4, 1, 0.6),
    mgp = c(2.4, 1, 0), las = 1)
plot(shortyears, log.shortair, type = "l", xlab = "Year",
     ylab = "log10(passenger numbers)", xlim = range(yr), ylim = c(2, 2.9))
mtext("SARIMA(1,1,0) x (1,0,0)")
## plots the forecasts
lines(fyears, forecast1, lwd = 2, col = "blue")
## plot the 95% prediction intervals.
lines(fyears, forecast1 + flimits1, col = "blue")
lines(fyears, forecast1 - flimits1, col = "blue")
points(fyears, log.airpass[133:144], pch = 16, cex = 0.5)

plot(shortyears, log.shortair, type = "l", xlab = "Year",
     ylab = "log10(passenger numbers)", xlim = range(yr), ylim = c(2, 2.9))
mtext("SARIMA(0,1,0) x (1,0,0)")
## plots the forecasts
lines(fyears, forecast2, lwd = 2, col = "red")
## plot the 95% prediction intervals.
lines(fyears, forecast2 + flimits2, col = "red")
lines(fyears, forecast2 - flimits2, col = "red")
points(fyears, log.airpass[133:144], pch = 16, cex = 0.5)

plot(shortyears, 10^log.shortair, type = "l", xlab = "Year",
     ylab="1000s of airline passengers", xlim = range(yr), ylim = c(100, 800))
lines(fyears, 10^forecast1, lwd = 2, col = "blue")
lines(fyears, 10^(forecast1 + flimits1), col = "blue")
lines(fyears, 10^(forecast1 - flimits1), col = "blue")
points(fyears, airpass[133:144], pch = 16, cex = 0.5)

plot(shortyears, 10^log.shortair, type = "l", xlab = "Year",
     ylab="1000s of airline passengers", xlim = range(yr), ylim = c(100, 800))
lines(fyears, 10^forecast2, lwd = 2, col = "red")
lines(fyears, 10^(forecast2 + flimits2), col = "red")
lines(fyears, 10^(forecast2 - flimits2), col = "red")
points(fyears, airpass[133:144], pch = 16, cex = 0.5)
```



Evaluate forecasts on the held-out year

Evaluate the same 12 observations for both models. RMSE measures error magnitude; signed mean relative error measures directional bias and can hide cancellation. We also report mean absolute relative error and interval coverage. Higher coverage is not automatically better: inspect interval width as well, especially with only 12 evaluation observations.

```
actual <- as.numeric(airpass[133:144])

forecast.metrics <- function(prediction, limits, actual) {
  point <- as.numeric(10^prediction)
  lower <- as.numeric(10^(prediction - limits))
  upper <- as.numeric(10^(prediction + limits))
  c(RMSE = sqrt(mean((point - actual)^2)),
    MRE = mean((point - actual) / actual),
    MAError = mean(abs(point - actual) / actual),
    Coverage = mean(actual >= lower & actual <= upper),
    MeanIntervalWidth = mean(upper - lower))
}

results <- rbind(
  "Nonseasonal + seasonal AR" = forecast.metrics(forecast1, flimits1, actual),
  "Seasonal AR only" = forecast.metrics(forecast2, flimits2, actual)
)
knitr::kable(results, digits = 3)
```

```
## Warning: 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
```

```
## See help("Deprecated")

## Warning: 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")
```

	RMSE	MRE	MARError	Coverage	MeanIntervalWidth
Nonseasonal + seasonal AR	30.364	0.057	0.057	0.917	169.128
Seasonal AR only	31.324	0.060	0.060	1.000	215.833

The point forecasts evaluated here are the exponentiated log forecasts, preserving the original session's calculation. If the goal is an original-scale mean forecast, use the Gaussian adjustment below rather than interpreting a median as a mean.

```
# Optional mean forecasts under the Gaussian log-scale approximation.
mean.forecast1 <- 10^forecast1 * exp(0.5 * log(10)^2 * preds1$se^2)
mean.forecast2 <- 10^forecast2 * exp(0.5 * log(10)^2 * preds2$se^2)
```

What to remember

- Transformations and differencing serve different purposes.
- Ordinary and seasonal ARMA terms describe dependence after differencing.
- Forecast differences and reconstruct levels; integrated-model software does this reconstruction internally.
- Check residual dependence, variability, and distributional assumptions.
- Evaluate point forecasts and prediction intervals on genuinely held-out data.